Kubernetes

Mise en Oeuvre

Press Space for next page →

Table des matières

- Introduction 3
- Installation
- Configuration
- Les Pods
- Le Scheduler
- Les Services
- Les deploiements
- Les Namespaces
- Les Volumes
- ConfigMap
- Secret

Support de cours Kubernetes.

Introduction
Historique

Historique

- Kubernetes ou k8s
- Borg de Google
- Cloud Native Computing Foundation (CNCF)
- Version 1.0 07/2015
- Version 1.35 12/2025

Qu’est ce que Kubernetes

- Orchestrateur
- Gestion d’applications conteneurisées
  - Déploiment
  - Self-healing
  - Montée en charge
  - Mise à jour
- Gestion des configurations
- Gestion des mots de passes (Secrets)
- RBAC

Concepts de base / Cluster

Cluster Kubernetes

- Ensemble de Nodes
- Linux ou Windows(1.14)

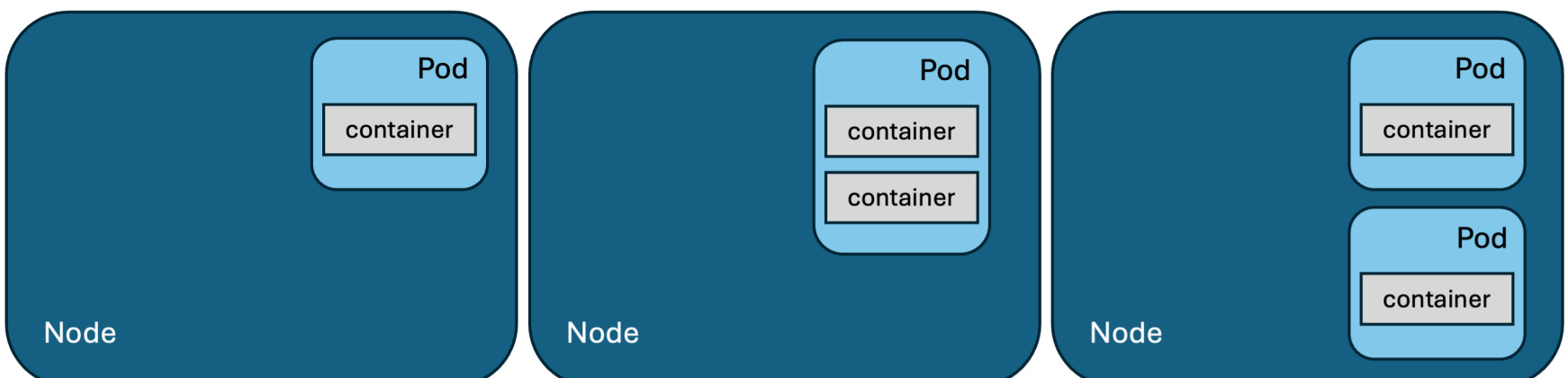
Concepts de base / Pods

Pods

- Ensemble de conteneurs
- Partage Réseau
- Partage Stockage

pod.yml

```yaml
kind: Pod
spec:
  containers:
  - name: web
```



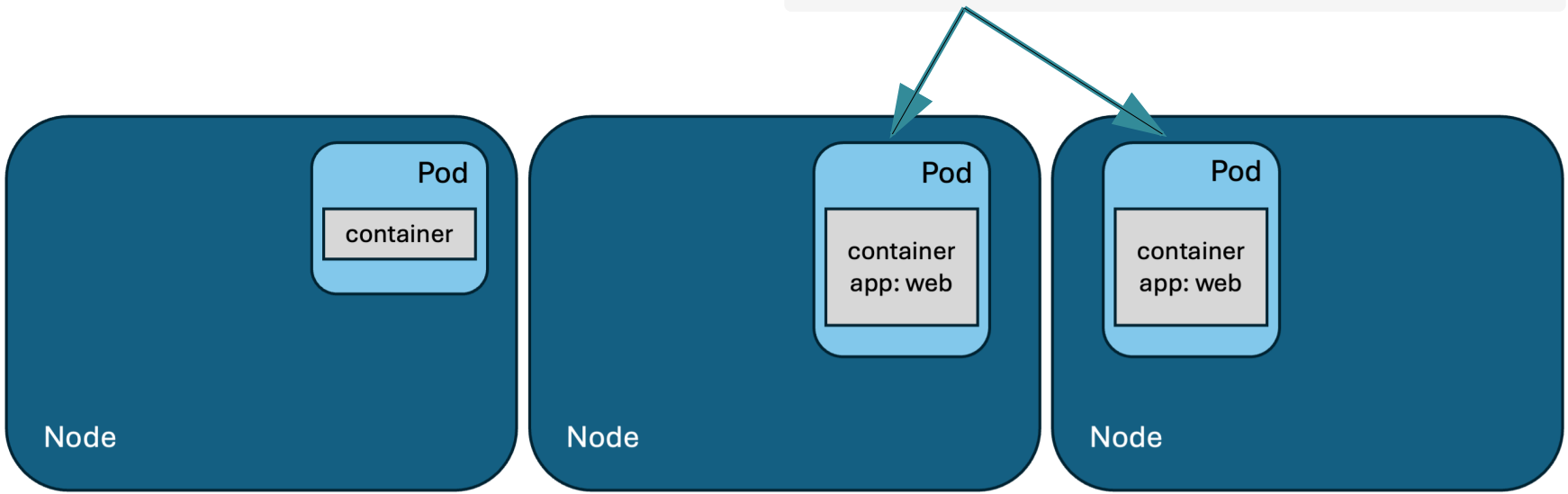
Concepts de base / Deployment

Deployment

- Définit la gestion d'un Pod
- Replicas
- Mise à Jour

deploy.yml

```yaml
kind: Deployment
spec:
  replicas: 2
```



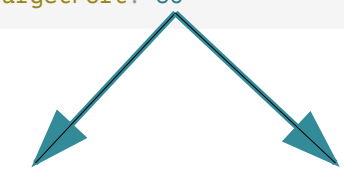
Concepts de base / Service

Service

- Permet d'exposer les ports d'un Pod
- A l'intérieur du Cluster
- A l'extérieur du Cluster

service.yml

```yaml
kind: Service
spec:
  ports:
  - port: 8080
    targetPort: 80
```



Concepts de base / Clients

Clients

- CLI : `kubectl`
- BUI : `dashboard`
- Fichier : `config`
- Gestion des Accès `Users/Access`

```
$ kubectl

$ ls ~/.kube
config
```

Concepts de base / Nodes

Master

- Gestion du cluster
- Orchestre l'exécution des Pods sur les Nodes
- Expose l'API server

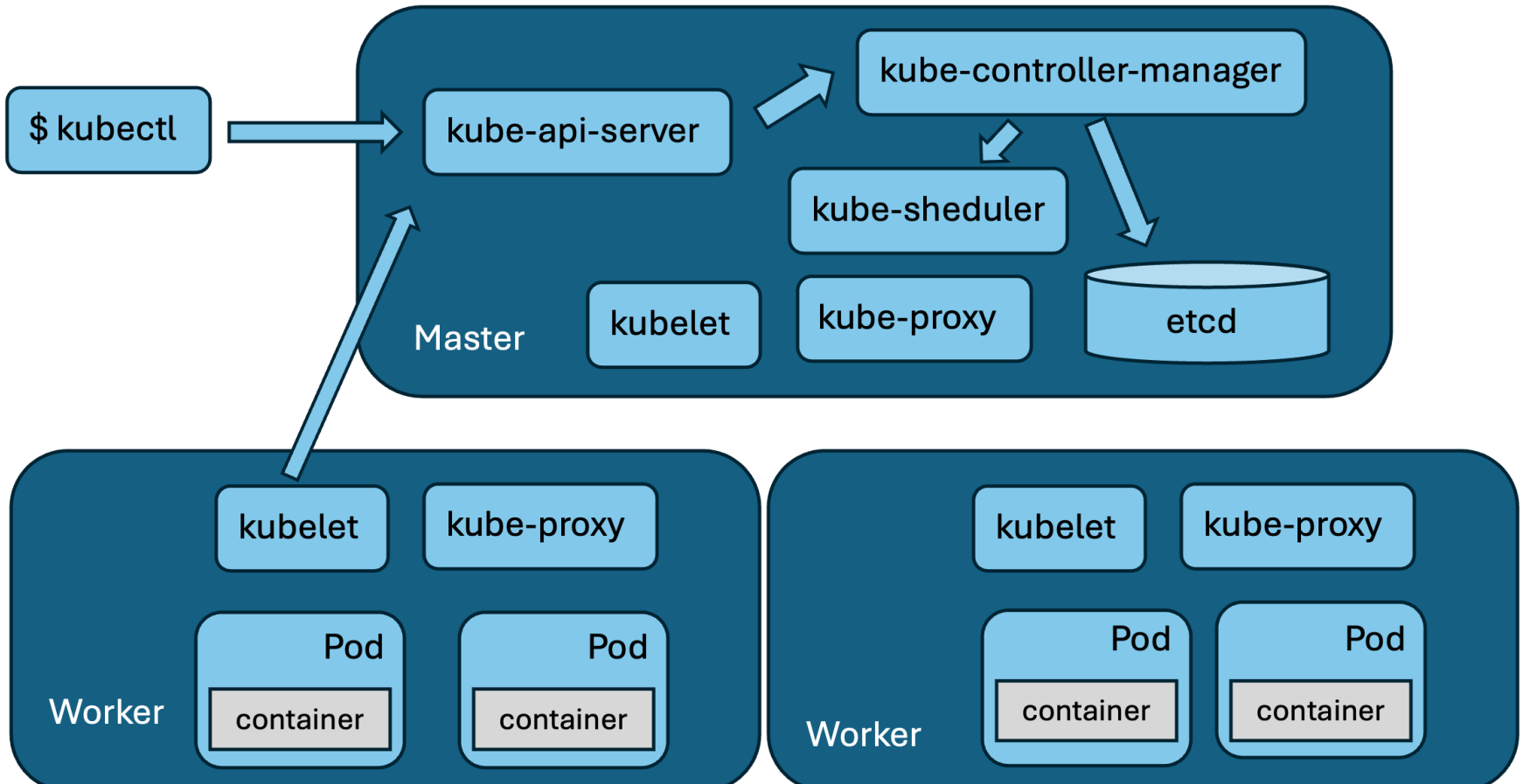
Worker

- Exécute les Pods du cluster
- Fournit les ressources aux Pods
- Communique avec le Master

```
$ kubectl get nodes
```

```
NAME     STATUS   ROLES           AGE     VERSION
host11   Ready    control-plane   7m2s    v1.34.3
host12   Ready    <none>          6m28s   v1.34.3
host13   Ready    <none>          6m28s   v1.34.3
```

Concepts de base / Process



Installation
GUIDE

Types d’installation

Solutions On-Premise

- kubeadm
- kubespray
- OpenShift (RedHat)
- Rancher (SUSE)
- Docker EE (Mirantis)
- Tanzu (VMware)

Solutions hébergées

- EKS : Amazon Elastic Kubernetes Service
- GKE : Google Kubernetes Engine
- AKS : Azure Kubernetes Service

Solutions locales

- Minikube
- Docker Desktop (Mac & Windows & Linux)

Environnement de tests

- play-with-k8s.com

minikube sur un Desktop

Solution tout en un

- Locale
- Orientée développeur

Basée sur une VM

- Hyperviseur Host-Based
- VirtualBox
- Vmware

Sur poste de travail

- Linux
- Windows
- Mac

pré-requis minikube

Installer un hyperviseur en local

Installer le binaire minikube

- Linux : https://github.com/kubernetes/minikube
- Windows : https://github.com/kubernetes/minikube/minikube-installer.exe
- MacOS : https://storage.googleapis.com/minikube/minikube-darwin-amd64

Installer le binaire kubectl

- https://kubernetes.io/fr/docs/tasks/tools/install-kubectl/

Docker Desktop

Basée sur un VM linux avec l'hyperviseur Natif

- Hyper-V à partir de Windows 10 Pro et Entreprise
- Hyperkit (xhyve) à partir de macOS Sierra 10.12
- Qemu sur Desktop Linux (Ubuntu …)

Activer Kubernetes comme solution Cluster

kubeadm

- Binaire kubeadm installé sur tous les nœuds du Cluster
- Docker ou autre Container Run Time installé

Master :

```
master# kubeadm init
master$ kubectl apply -f https://raw.githubusercontent.com/coreos/kube-flannel.yml
```

Worker :

```
worker# kubeadm join 192.168.116.160:6443 --token mm20xq.goxx7plwzrx75tv3
```

Configuration & Contexts

Context

Context d’utilisation

- Définit le cluster k8s cible
- Définit l’utilisateur

Plusieurs contexts peuvent être disponibles

- Un seul context actif indiqué par CURRENT *

```
$ kubectl config get-contexts
```

```
CURRENT   NAME                          CLUSTER         AUTHINFO           NAMESPACE
*         kubernetes-admin@cluster.local   cluster.local   kubernetes-admin
```

Configuration

Context utilisé par kubectl dans l'ordre de priorité

1. Option de la commande

   `--kubeconfig`

   spécifie un fichier de config

2. Variable d'environnement
   $KUBECONFIG

3. Fichier kubeconfig par défaut
   $HOME/.kube/config

Context de configuration

```
$ kubectl config view
```

```
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://host11:6443
  name: cluster.local
contexts:
- context:
    cluster: cluster.local
    user: kubernetes-admin
  name: kubernetes-admin@cluster.local
current-context: kubernetes-admin@cluster.local
kind: Config
preferences: {}
users:
- name: kubernetes-admin
  user:
    client-certificate-data: DATA+OMITTED
    client-key-data: DATA+OMITTED
```

Commandes de Context

Lister les contexts

```
$ kubectl config get-contexts
```

```
CURRENT   NAME                          CLUSTER         AUTHINFO           NAMESPACE
*         kubernetes-admin@cluster.local   cluster.local   kubernetes-admin
          minikube                      minikube        minikube
```

Changer de context

```
$ kubectl config use-context minikube
```

```
Switched to context "minikube"
```

Supprimer un context

```
$ kubectl config delete-context minikube
```

```
deleted context minikube from /Users/samir/.kube/config
```

Les Pods

Catégories de ressources

Gestion des Applications

- Pod
- ReplicaSet
- Deployment

Load balancing

- Service

Configuration des Applications

- ConfigMap
- Secret

Stockage

- Persistent Volume
- Persistent Volume Claim

Configuration du Cluster

- Metadata
- Namespace
- Roles

Clés de définition

apiVersion

- v1
- apps/v1

kind

- Pod
- Deployment

metadata

- name
- labels

spec

- containers
- replicas

exemple

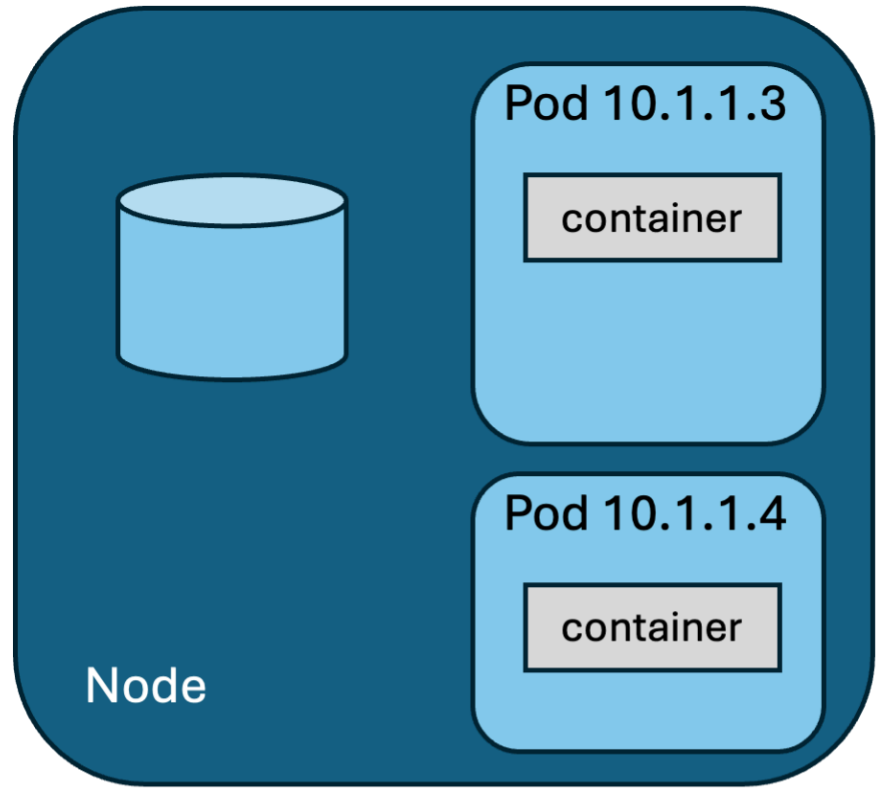
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: www
  image: nginx:1.12.2
```

Pod

Unité de déploiement applicative
Ensemble de conteneurs

- même contexte d'isolation
- même stack réseau
- mêmes volumes partagés
- même adresse IP dédiée
- des spécification d'exécution

Ephémères
Best Practice : 1 Pod = 1 Container

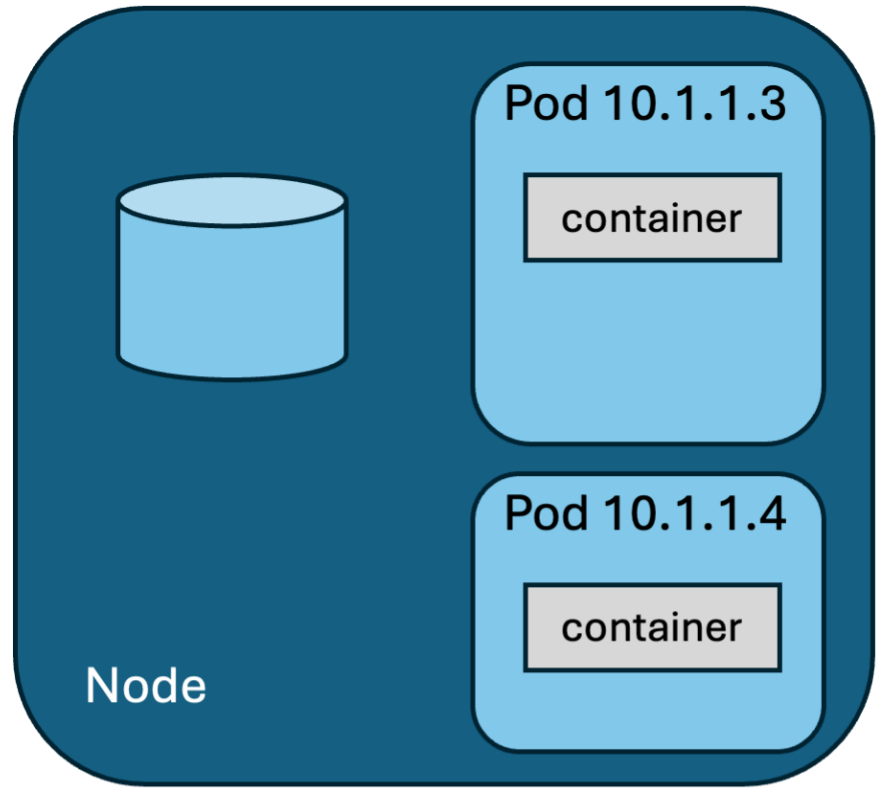


Pod

Une Application

- découpée en 1 ou plusieurs Pods
- chaque Pod fournit un service

Scalling horizontal

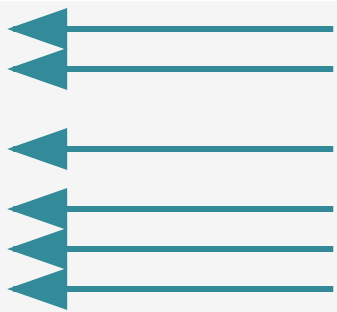


Exemple de Spécification

Spécification au format YML ou JSON

```
$ cat web.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: www
    image: nginx:1.17
```



Version de l'API
Type de l'objet

Nom de l'objet
Spécifications des conteneurs
Nom du premier conteneur
Image du premier conteneur

```
  - name: debug
    image: alpine:3.12
```

Deuxième conteneur

Création d'un Pod

Créer un Pod

```
kubectl create -f SPEC.yml
kubectl apply -f SPEC.yml
```

```
$ kubectl apply -f  web.yml
```

```
pod/web created
```

Lister les Pods

```
kubectl get pods
```

```
$ kubectl get pod
```

```
NAME  READY   STATUS    RESTARTS    AGE
web   1/1     Running   0           41s
```

Pod / Etats

Etas d’un Pod

```
Pending
```

Le Pod est en attente (manque une affinité)

```
Running
```

Le Pod est en exécution

```
$ kubectl get po
```

```
NAME  READY STATUS            RESTARTS  AGE
web   0/1   ContainerCreating 0         2s
```

```
CrashLoopBackOff
```

Trop de restart

```
CrashPullBackOff
```

Erreur de pull

```
Succeeded
```

Pod terminé avec succès, pas de restart

```
Failed
```

Pod terminé avec erreur, pas de restart

Suppression d’un Pod

Supprimer un Pod

```
kubectl delete pod NAME
```

```
$ kubectl delete po web
```

```
pod "web" deleted
```

Lister les Pods

```
kubectl get pods
```

```
$ kubectl get pod
```

```
No resources found.
```

Description d’un Pod

```
kubectl describe pod NAME
```

```
$ kubectl describe po web
```

```
Name:             web
Namespace:        default
Priority:         0
Service Account:  default
Node:             host102/192.168.1.102
Start Time:       Sun, 29 Mar 2026 18:39:30 +0200
Labels:           <none>
Annotations:      cni.projectcalico.org/containerID: 479d6a7e1fc88bdb5a6c0a5fc8280ea90d4c3edc3fa92c73929e779d0a835f27
                  cni.projectcalico.org/podIP: 10.233.117.194/32
                  cni.projectcalico.org/podIPs: 10.233.117.194/32
Status:           Running
IP:               10.233.117.194
IPs:
  IP:  10.233.117.194
Containers:
  www:
    Container ID:   containerd://017e7c5d33af240ed2ed8f1050f1e349552dd8a6ffa9907f15eec7141df43e9f
    Image:          nginx:1.17
```

Description d’un Pod

Description (suite)

```
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  16s   default-scheduler  Successfully assigned default/web to host102
  Normal  Pulling    16s   kubelet            Pulling image "nginx:1.17"
  Normal  Pulled     12s   kubelet            Successfully pulled image "nginx:1.17" in 4.451s (4.451s including waiting).
  Normal  Created    12s   kubelet            Created container: www
  Normal  Started    12s   kubelet            Started container www
```

Logs

Logs d’un Pod

```
kubectl logs [-f] pod/NAME [-c CONTAINER]
```

```
$ kubectl logs -f po/web -c www
```

```
127.0.0.1 - - [29/Mar/2026:16:46:58 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.76.1" "-"
127.0.0.1 - - [29/Mar/2026:16:47:24 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.76.1" "-"
127.0.0.1 - - [29/Mar/2026:16:47:25 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.76.1" "-"
127.0.0.1 - - [29/Mar/2026:16:47:28 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.76.1" "-"
```

Exécution d’une commande

```
kubectl exec [-it] pod/NAME [-c CONTAINER] -- COMMAND
```

```
$ kubectl exec -it web -- bash
```

```
root@web:/#
```

Lorsque le Pod contient plusieurs conteneurs

```
$ kubectl exec -it web -- bash
```

```
Defaulted container "www" out of: www, debug
root@web:/#
```

```
$ kubectl exec -it web -c debug -- bash
```

```
root@web:/#
```

Port Forwarding

Publie le port d'un Pod sur le poste de contrôle
Plutôt utilisée en mode debug/test ou en developpement

```
kubectl port-forward POD Host_PORT:CONTAINER_PORT
```

```
$ kubectl port-forward web 8000:80
```

```
Forwarding from 127.0.0.1:8000 -> 80
Forwarding from [::1]:8000 -> 80
Handling connection for 8000
```

```
$ curl 127.0.0.1:8000
```

```
<!DOCTYPE html>
<html>
<title>Welcome to nginx!</title>
```

Impérative vs Déclarative

Approche impérative

- Expliciter les étapes à exécuter pour créer l’objet final

Approche déclarative

- Déclarer que est le résultat final désiré, sans indiquer le processus pour y parvenir

Impérative

Créer un objet en ligne de commande

```
$ kubectl run --image=nginx:1.21-alpine web
```

Créer un objet avec un fichier de spécification

```
$ kubectl create -f nginx.yml
```

Modifier un objet avec un fichier de spécification

```
$ kubectl replace -f nginx.yml
```

Déclarative

Créer un objet avec un fichier de spécification

```
$ kubectl apply -f nginx.yml
```

Modifier un objet avec un fichier de spécification

```
$ kubectl apply -f nginx.yml
```

Documentation

https://kubernetes.io/docs/tasks/manage-kubernetes-objects/declarative-config/

Déclarative/apply

Fichier Local

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: www
    image: nginx:1.17
```

dernière configuration appliquée

```json
{
    "kind": "Pod",
    "apiVersion": "v1",
    "metadata": {
        "name": "web",
        "labels": {
            "run": "web"
        }
    },
    "spec": {
        "containers": [
            {
                "name": "web",
                "image": "nginx:1.17",
                "resources": {}
            }
        ],
        "restartPolicy": "Always",
    },
    "status": {}
}
```

Live Kubernetes

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: www
    image: nginx:1.17
```

Scheduler

Labels

Paires Clé=Valeur
Attachées aux Objets Kubernetes

- Pods
- Services
- Déploiement
- Nœuds

Organiser les applications sur le Cluster
Créés lors du déploiement ou ultérieurement
Modifiés à tout moment

- ”release” : ”stable” , ”release” : ”test”
- ”environment” : ”dev” , ”environment” : ”production”

Ajouts de Labels

Créer un Label

```
kubectl label < type > < object > label_name=value
```

Créer un Label sans valeur

```
kubectl label < type > < object > label_name=
```

Supprimer un Label

```
kubectl label < type > < object > label_name-
```

Ajouts de Labels (exemples)

Créer un Label

```
$ kubectl label node host13 app.dom.com/disktype=ssd
```

```
node/workhost11 labeled
```

Visualiser les Labels

```
$ kubectl get nodes --show-labels
```

```
NAME     STATUS   ROLES           AGE     VERSION   LABELS
host11   Ready    control-plane   4m35s   v1.34.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io
host12   Ready    <none>          4m1s    v1.34.3   beta.kubernetes.io/arch=amd64,beta.kubernetes.io/os=linux,kubernetes.io
host13   Ready    <none>          4m1s    v1.34.3   app.dom.com/disktype=ssd,beta.kubernetes.io/arch=amd64,beta.kubernetes.
```

Selecteurs

Sélection d’objets à partir de Labels

Identifie un ensemble spécifique d’objets

Deux types

- Equality-based
  - = égalité stricte
  - != différence
- Set-based
  - IN la valeur est dans un ensemble de valeurs définies
  - NOTIN la valeur ne doit pas figurer dans un ensemble de valeurs définies
  - EXISTS détermine seulement si le Label existe ou pas

nodeName

Permet de réaliser un scheduling manuel

```
$ cat web.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: www
    image: nginx:1.17
  nodeName: host13
```

nodeSelector

Permet de lancer un Pod sur un nœud ayant un Label spécifique

```
$ cat web.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: www
    image: nginx:1.17
  nodeSelector:
    disktype: ssd
```

nodeAffinity

Permet de lancer un Pod sur un nœud ayant une affinité

```yaml
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
            - worker1
            - worker2
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
```

podAffinity

- Schéduler un pod par affinité positive avec les autres pods
- Règles hard ou soft
- Clé topologyKey : spécifie le domaine d'exécution (hostname,region,zone)

```
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: security
            operator: In
            values:
            - S1
        topologyKey: kubernetes.io/zone
```

podAntiAffinity

- Schéduler un pod par affinité négative avec les autres pods
- Règles hard ou soft
- Clé topologyKey : spécifie le domaine d'exécution (hostname,region,zone)

```yaml
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: security
              operator: In
              values:
              - S2
          topologyKey: kubernetes.io/hostname
```

Taints

- Taint est l'inverse de l'affinité
  - Déclaré sur un noeud
  - Permet à un nœud de repousser un pod

```
kubectl taint nodes < node_name > key=value:action
```

- Action

```
- NoSchedule
- PreferNoSchedule
- NoExecute
```

```
$ kubectl taint nodes host11 app=db:NoSchedule
```

```
node/host11 tainted
```

Tolerations

- Les tolérances sont appliquées aux pods
- Permettent aux pods de tolérer un Taint et d'êtres schédulés sur ces nœuds

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: myapp
spec:
  tolerations:
  - key: "app"
    operator: "Equal"
    value: "db"
    effect: "NoSchedule"
```

Complétion

- Completion
  - Commandes, sous-commandes et options
  - ressources

```
$ sudo yum install -y bash-completion
```

```
  Installé :
  bash-completion.noarch 1:2.1-6.el7
```

```
cat ~/.bashrc
```

```
  source <(kubectl completion bash)
```

```
kubectl delete pods <tab>
```

```
  web debug
```

Ressources

- La commande `kubectl api-resources` permet de lister les ressources accessibles par l'API

```
$ kubectl api-resources
```

```
NAME                              SHORTNAMES   APIVERSION                     NAMESPACED   KIND
bindings                                       v1                             true         Binding
componentstatuses                 cs           v1                             false        ComponentStatus
configmaps                        cm           v1                             true         ConfigMap
endpoints                         ep           v1                             true         Endpoints
namespaces                        ns           v1                             false        Namespace
nodes                             no           v1                             false        Node
persistentvolumeclaims            pvc          v1                             true         PersistentVolumeClaim
persistentvolumes                 pv           v1                             false        PersistentVolume
pods                              po           v1                             true         Pod
resourcequotas                    quota        v1                             true         ResourceQuota
secrets                                        v1                             true         Secret
serviceaccounts                   sa           v1                             true         ServiceAccount
services                          svc          v1                             true         Service
controllerrevisions                            apps/v1                        true         ControllerRevision
daemonsets                        ds           apps/v1                        true         DaemonSet
deployments                       deploy       apps/v1                        true         Deployment
replicasets                       rs           apps/v1                        true         ReplicaSet
```

Ressources Documentation

- La commande `kubectl explain` permet d'avoir la documentation des ressources

```
$ kubectl explain pod.spec.affinity
```

```
KIND:       Pod
VERSION:    v1

FIELD: affinity <Affinity>

DESCRIPTION:
    If specified, the pod's scheduling constraints
    Affinity is a group of affinity scheduling rules.

FIELDS:
  nodeAffinity  <NodeAffinity>
    Describes node affinity scheduling rules for the pod.

  podAffinity <PodAffinity>
    Describes pod affinity scheduling rules (e.g. co-locate this pod in the same
    node, zone, etc. as some other pod(s)).

  podAntiAffinity <PodAntiAffinity>
    Describes pod anti-affinity scheduling rules (e.g. avoid putting this pod in
    the same node, zone, etc. as some other pod(s)).
```

Mode Dry Run

- La commande `kubectl --dry-run` permet de simuler une commande sans modifications

```
$ kubectl run --image=nginx web1 --restart=Never --dry-run=client -o yaml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: web1
  name: web1
spec:
  containers:
  - image: nginx
    name: web1
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Never
status: {}
```

Format JSON

- Template JSONPATH `https://kubernetes.io/docs/reference/kubectl/jsonpath`

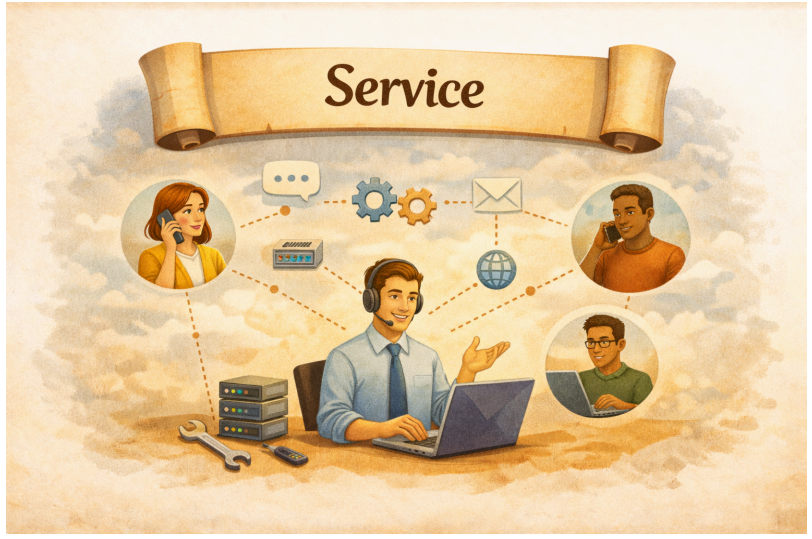
```
$ kubectl get pods -o json
$ kubectl get pods -o yaml

$ kubectl get pods -o jsonpath='{.items[0]}'
$ kubectl get pods -o=jsonpath='{.items[0].metadata.name}'

$ kubectl get po/web1 -o jsonpath='{.spec}'
```

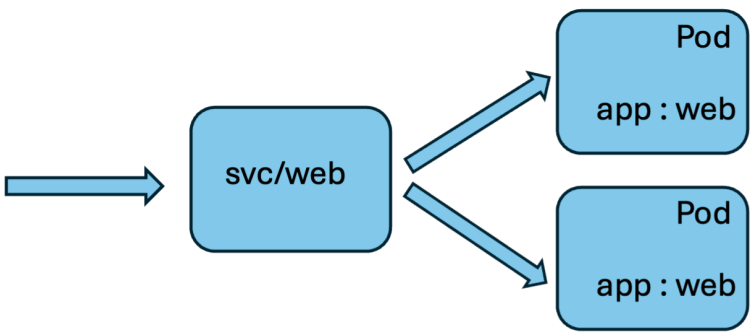
```
$ kubectl get pods -o=jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.startTime}{"\n"}{end}'
```

```
web1  2026-04-17T10:58:55Z
web2  2026-04-17T10:58:56Z
web3  2026-04-17T10:58:58Z
```


Service

Service

- Gère un ensemble de Pods
- Regroupés à partir de Labels
- Load-Balancing
- Possède une adresse IP privée résolue par un DNS interne



Types de Service

- ClusterIP
  - Expose à l'intérieur du Cluster
  - Accès via le nom du Service
- NodePort
  - Expose à l'extérieur du cluster
  - Accès via des Ports sur les hosts du Cluster
- LoadBalancer
  - Ingress Controlrer
  - Spécifique aux Cloud Provider
- ExternalName
  - Alias vers un Service externe au Cluster

ClusterIP

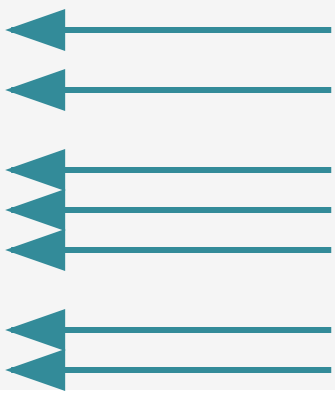
- Expose à l'intérieur du Cluster
- Accès via le nom du Service
- Algorithme Random
- Session Affinity
  - `None`: par défaut, pas d'affinité entre le client et le serveur (pod)
  - `ClientIP`: affinité entre le client et le serveur

ClusterIP

- Spécification du Service

```
$ cat web-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web
  type: ClusterIP
  ports:
  - port: 80
    targetPort: 80
```



Type de l'objet
Nom de l'objet service
Selecteur de groupe de Pods
Clé: Valeur du selecteur
type du service
port exposé dans le cluster
port cible d’un Pod du groupe

ClusterIP

- Spécification du Pod à regrouper

```
$ cat web.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: www
  labels:
    app: web
spec:
  containers:
  - name: www
    image: nginx:1.17
```

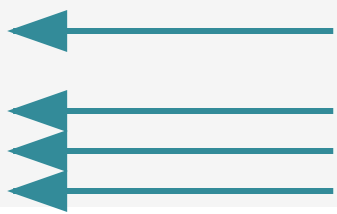
Label de regroupement

NodePort

- Spécification du Service

```
$ cat web-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web
  type: NodePort
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30000
```



type du service
port exposé dans le cluster
port cible d'un Pod du groupe
port exposé par les noeuds à l'extérieur du cluster

Deployment

ReplicaSet

- Spécification du Service
  - ReplicaSet
  - gère des Pods de même spécification
  - s’assure de l’état et du nombre souhaité de replicas
- Load Balancing
- Scaling

ReplicaSet

- Spécification du ReplicaSet

```
$ cat myapp-rs.yml
```

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      name: myapp
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:1.21-alpine
```

sélection par le Label des Pods gérés par le ReplicaSet

label du pod

ReplicaSet

```
$ kubectl apply -f myapp-rs.yml
```

```
replicaset.apps/myapp created
```

```
$ kubectl get all -o wide --show-labels
```

```
NAME              READY   STATUS    RESTARTS   AGE     IP               NODE     NOMINATED NODE   READINESS GATES   LABELS
pod/myapp-f44k9   1/1     Running   0          2m43s   10.233.70.133    host13   <none>           <none>            app=mya
pod/myapp-lx4xt   1/1     Running   0          2m43s   10.233.89.65     host11   <none>           <none>            app=mya
pod/myapp-prbxh   1/1     Running   0          2m43s   10.233.125.196   host12   <none>           <none>            app=mya

NAME                    DESIRED   CURRENT   READY   AGE     CONTAINERS   IMAGES              SELECTOR    LABELS
replicaset.apps/myapp   3         3         3       2m43s   myapp        nginx:1.21-alpine   app=myapp   <none>
```

ReplicaSet

- Selector obligatoire
  - FIELDS:

    `matchExpressions []Object`

    `matchLabels map[string]string`

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      env: prod
      ver: "1.2"
    matchExpressions:
    - { key: tier, operator: In, values: [frontend] }
    - { key: ver, operator: In, values: ["1.2","1.3"] }
```

Scale

- Modification du fichier de description et mise à jour

```
kubectl replace –f rs-definition.yml
```

- Commande `scale` et fichier yaml

```
kubectl scale --replicas=new-value –f rs-definition.yml
```

- Commande `scale` et arguments

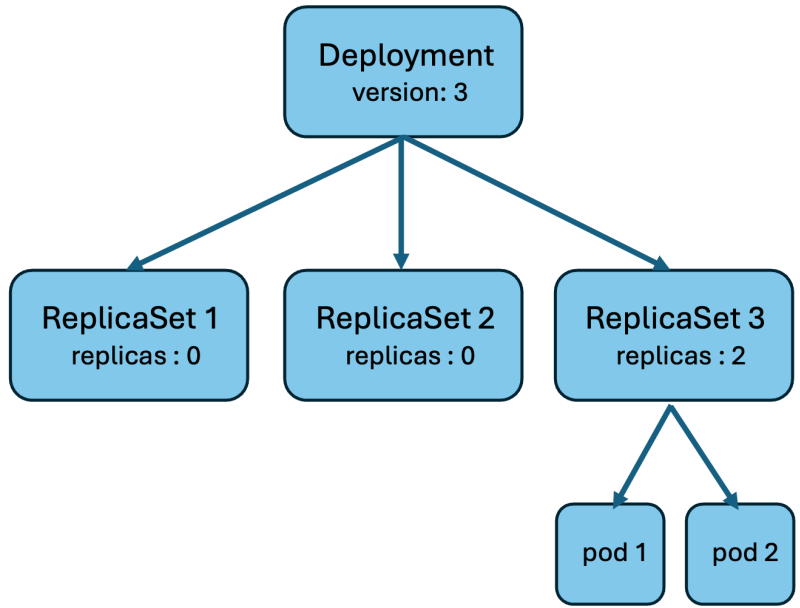
```
kubectl scale --replicas=new-value replicaset myapp-rs
```

- Commande `patch`

```
kubectl patch rs myapp-rs –p '{"spec":{"replicas":6}}'
```

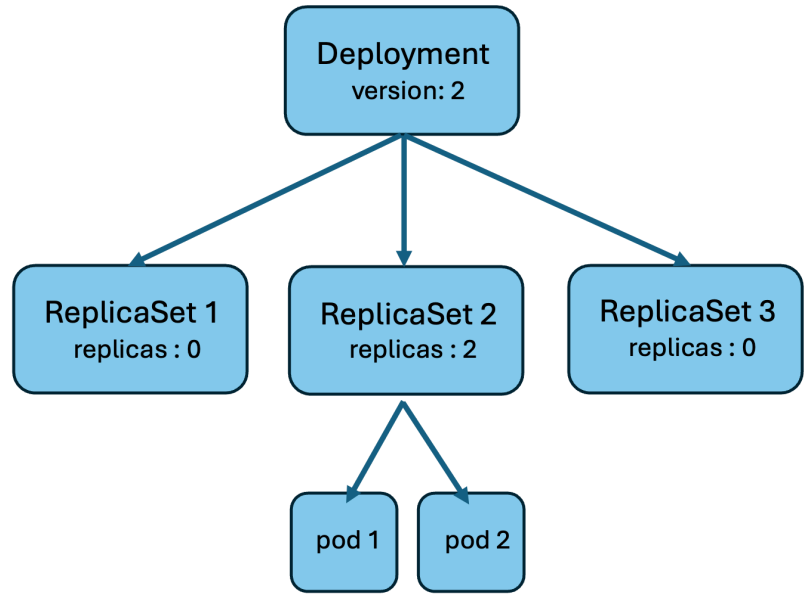
Deployment

- Deployment
  - haut niveau d'abstraction
  - gère des ReplicaSet
- ReplicaSet
  - gère des Pods de même spécification
  - s'assure de l'état souhaité des Pods
- Pod instance applicative
  - géré par un ReplicaSet
  - schédulé sur un node du Cluster



Deployment

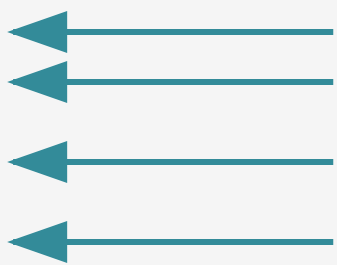
- Défini un état souhaité
  - suivant la version d'un Pod
  - du nombre de réplicas désiré
- Associe un ReplicatSet
  - assure de l'état désiré
- Gère les mise à jour des applications
  - rolling update
  - rollback
  - scaling



Deployment

```
$ cat web-deployment.yml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  labels:
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.17
```



3 pods désirés

Selection par le Label des Pods gérés par le ReplicaSet

spécification des Pods

labels des Pods

Deployment

```
$ kubectl apply -f web-deployment.yml
```

```
deployment.apps/web-deployment created
```

```
$ kubectl get deployments
```

```
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
web-deployment   3/3     3            3           15s
```

```
$ kubectl get replicasets
```

```
NAME                        DESIRED   CURRENT   READY   AGE
web-deployment-67c68c959f   3         3         3       21s
```

```
$ kubectl get pods
```

```
NAME                              READY   STATUS    RESTARTS   AGE
web-deployment-67c68c959f-bgm6p   1/1     Running   0          29s
web-deployment-67c68c959f-rgfdn   1/1     Running   0          29s
web-deployment-67c68c959f-w9m6r   1/1     Running   0          29s
```

Scale

- Modification du fichier de description et mise à jour

```
kubectl replace –f deploy-definition.yml
```

- Commande `scale` et fichier yaml

```
kubectl scale --replicas=new-value –f deploy-definition.yml
```

- Commande `scale` et arguments

```
kubectl scale --replicas=new-value deploy myapp-deploy
```

- Commande `edit`

```
kubectl edit deploy myapp-deploy
```

Rolling Update

- Modification du fichier de description et mise à jour

```
kubectl replace –f deploy-definition.yml
```

- Modification par la commande `set`

```
kubectl set image deployment deploy_name container_name=new_image
```

- Modification par la commande `edit`

```
kubectl edit deployment deploy_name
```

- Suivre l'évolution de la mise à jour avec la commande `status`

```
kubectl rollout status -w deployment deploy_name
```

Historique

- Historique des mises à jour

```
kubectl rollout history deployment deploy_name
```

- Détails d'une version de l'historique

```
kubectl rollout history deployment deploy_name --revision=number
```

Rollback

- Modification du fichier de description et mise à jour

```
kubectl apply –f deploy-manifest.yml
```

- La commande undo

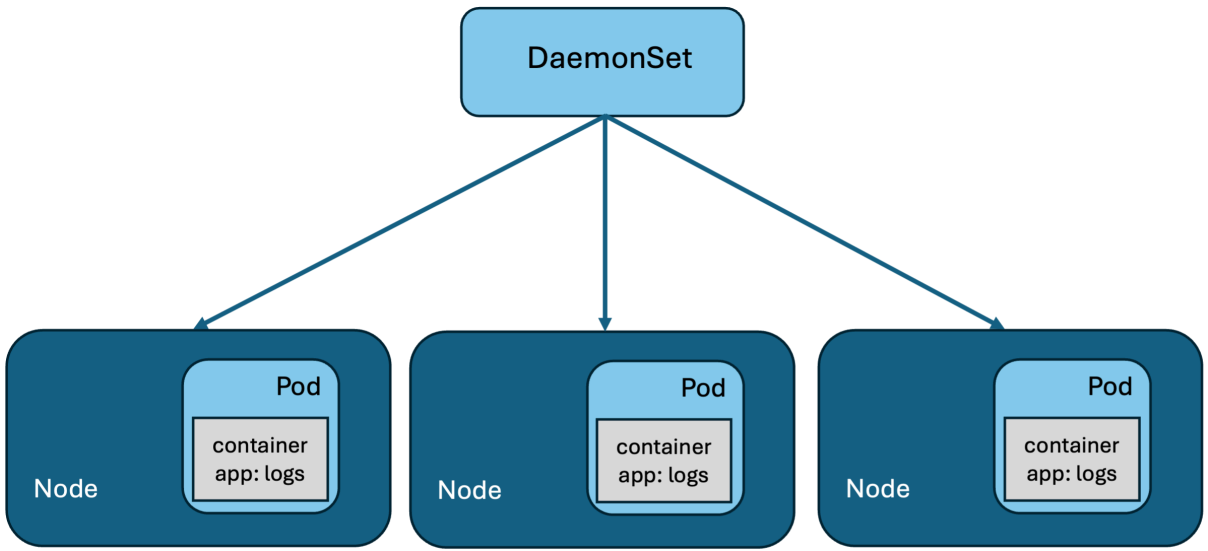
```
kubectl rollout undo deployment deploy_name
```

- Rollback vers une version spécifique

```
kubectl rollout undo deployment deploy_name --to-revision=number
```

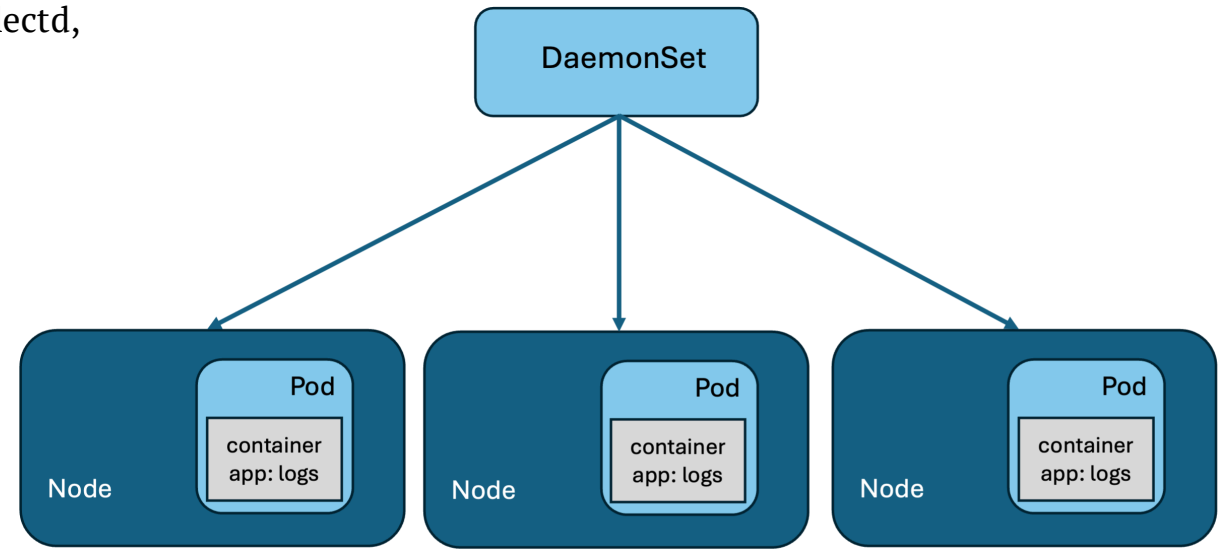
DaemonSet

- Un DaemonSet garantit que tous les nœuds exécutent une copie du pod.
- Lorsque des nœuds sont ajoutés au cluster, des pods leur sont ajoutés.



DaemonSet/Use Case

- Exécuter un démon de stockage en cluster, tel que glusterd, ceph, sur chaque nœud.
- Exécuter un démon de collecte de journaux sur chaque nœud, tel que fluentd ou filebeat.
- Exécuter un démon de surveillance sur chaque nœud, tel que Prometheus Node Exporter, Flowmill, Sysdig Agent, collectd,



DaemonSet/Spécification

```
$ cat web-ds.yml
```

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx:1.17-alpine
```

DaemonSet/Spécification

```
$ kubectl apply -f web-daemonset.yml
```

```
daemonset.apps/web created
```

```
$ kubectl get daemonsets
```

```
NAME   DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
web    3         3         3       3            3           <none>          2m15s
```

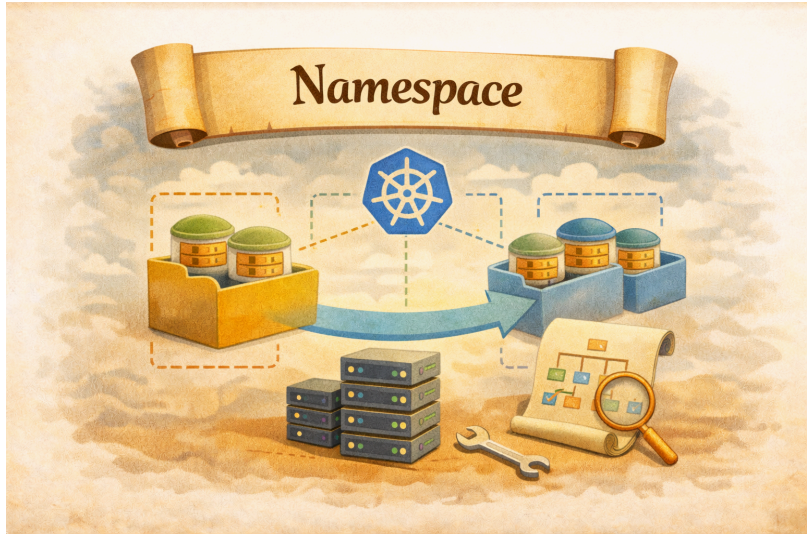
```
$ kubectl get pods
```

```
NAME        READY   STATUS    RESTARTS   AGE
web-4mvs6   1/1     Running   0          3m15s
web-4nj8x   1/1     Running   0          3m15s
web-l6w6p   1/1     Running   0          3m15s
```

Rolling Update et Rollback

- Identique au deployment
- Stratégie de mise à jour
  - spec: `updateStrategy`:
    - type: RollingUpdate (valeur par défaut)

    `RollingUpdate` : les pods sont mis à jour en même temps que le DaemonSet
    `OnDelete` : les pods sont mis à jour suite à la commande delete


Namespace

Namespace

- Isoler les ressources
  - Pods
  - Services
  - Deployments
- Segmenter un Cluster
  - Departements
  - Projets
  - Clients

```
$ kubectl get namespaces
```

```
NAME              STATUS   AGE
default           Active   46h
kube-public       Active   46h
kube-system       Active   46h
```

Création

- En ligne de commande

```
$ kubectl create namespace production
```

```
namespace/production created
```

- A partir d'un fichier de spécification

```
$ cat production.yml
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: production
```

```
$ kubectl apply -f production.yml
```

```
namespace/production created
```

Désignation du Namespace 1/3

- Le namespace est assigné dans les metadatas du fichier de spécifications

```
$ cat web.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  namespace: production
spec:
  containers:
  - name: www
    image: nginx:1.17
```

- Création du Pod

```
$ kubectl apply -f web.yml
```

```
pod/web created
```

Désignation du Namespace 2/3

- Le namespace est assigné lors de la création en ligne de commande

```
$ cat web.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: www
    image: nginx:1.17
```

- Création du Pod

```
$ kubectl apply -f web.yml --namespace=production
```

```
pod/web created
```

Désignation du Namespace 3/3

- Le namespace est assigné dans le context

```
$ kubectl config set-context prod --cluster=cluster1 --user=kubernetes-admin --namespace=production
```

```
Context "prod" created.
```

```
$ kubectl config view
```

```
apiVersion: v1
clusters:
- cluster:
    server: https://host11:6443
  name: cluster1
contexts:
- context:
    cluster: cluster1
    user: kubernetes-admin
  name: kubernetes-admin@cluster1
- context:
    cluster: cluster1
    namespace: production
    user: kubernetes-admin
  name: prod
```

Ressource Quota

- Permet de limiter la consommation de certaines ressources
  - `limits.cpu:` CPU maximum consommée par l'ensemble des pods du namespace
  - `limits.memory:` RAM maximum consommée par l'ensemble des pods
  - `requests.cpu:` CPU minimum requise par l'ensemble des pods du namespace
  - `requests.memory:` RAM minimum requise par l'ensemble des pods du namespace

Ressource Quota

- Créer une ressource Quota

```
$ cat rq-dev.yml
```

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
spec:
  hard:
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

Ressource Quota

- Appliquer les quotas à un namespace

```
$ kubectl apply -f rq-dev.yml --namespace development
```

```
resourcequota/dev-quota created
```

- Lire les consommations des quotas

```
$ kubectl get resourcequota --namespace development
```

```
NAME        AGE   REQUEST                                      LIMIT
dev-quota   57s   requests.cpu: 0/1, requests.memory: 0/1Gi   limits.cpu: 0/2, limits.memory: 0/2Gi
```

Volumes
/data
BACKUP

Volumes

- Extérieur au container
- Cycle de vie indépendant
- Persistance des données
- Partage de volume entre containers d’un même Pod
- Plusieurs types disponibles
  - emptyDir
  - configMap
  - iscsi
  - nfs
  - cephfs

EmptyDir

- Lié au cycle de vie d'un Pod
- Vide à la création

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
  - name: mysql
    image: mysql:7.5
    volumeMounts:
    - name: data
      mountPath: /var/lib/mysql
  volumes:
  - name: data
    emptydir:
```

hostPath

- Monter un répertoire ou un fichier du hôte

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
  - name: mysql
    image: mysql:7.5
    volumeMounts:
    - name: data
      mountPath: /var/lib/mysql
  volumes:
  - name: data
    hostPath:
      path: /data/db
```

NFS

- Monter un partage NFS

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
  - name: mysql
    image: mysql:7.5
    volumeMounts:
    - name: data
      mountPath: /var/lib/mysql
  volumes:
  - name: data
    nfs:
      server: 192.168.1.1
      path: /data/db
```

Fibre Channel

- Volume de type SAN

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
  - name: mysql
    image: mysql:7.5
    volumeMounts:
    - name: data
      mountPath: /var/lib/mysql
  volumes:
  - name: data
    fc:
      targetWWNs:
      - XXXXXXXXXX
      lun: 0
      fstype: ext4
```

iSCSI

- Volume de type iSCSI

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
  - name: mysql
    image: mysql:7.5
    volumeMounts:
    - name: data
      mountPath: /var/lib/mysql
  volumes:
  - name: data
    iscsi:
      iqn: iqn.2001-08.com.netapp.xxxx
      lun: 0
      fstype: ext4
```

Persistent Volume

- Persistent Volume
  - Stockage provisionné
  - Statique
  - Dynamique
- Persistent Volume Claim
  - Demande de Stockage
  - Spécifie des Contraintes
    - taille, type …
  - Consomme un PV existant ou provisionnement PV par StorageClass
  - Référencé dans un Pod

Exemple statique 1/3

```
$ cat pv.yml
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv
spec:
  storageClassName: manual
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/pv
```

```
$ kubectl apply -f pv.yml
```

```
persistentvolume/pv created
```

```
$ kubectl get pv
```

```
NAME   CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      CLAIM   STORAGECLASS   REASON   AGE
pv     5Gi        RWO            Retain           Available           manual                  8s
```

Exemple statique 2/3

```
$ cat pvc.yml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

```
$ kubectl apply -f pvc.yml
```

```
persistentvolumeclaim/claim created
```

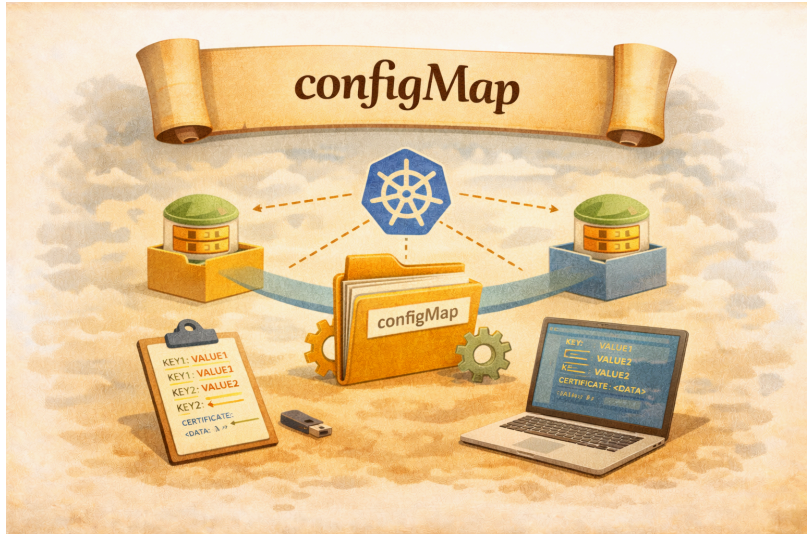
```
$ kubectl get pvc
```

```
NAME    STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
claim   Bound    pv       5Gi        RWO            manual         6s
```

Exemple statique 3/3

```
$ cat pod.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
  - name: mysql
    image: mysql:7.5
    volumeMounts:
    - name: data
      mountPath: /var/lib/mysql
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: claim
```


configMap
configMap
KEY1: VALUE1
KEY1: VALUE1
KEY2: VALUE2
KEY2:
CERTIFICATE:
<DATA>
KEY: VALUE1
VALUE1
K VALUE2
CERTIFICATE: <DATA>

ConfigMap

- Permet de détacher l'application de sa configuration
- Portabilité
- Créée à partir
  - d'un fichier
  - d'un répertoire
  - de valeurs littérales
- Constituée de
  - une ou plusieurs clé=valeur

Création

- A partir d'un fichier

```
$ cat nginx.conf
```

```
user  nginx;
worker_processes  auto;

error_log  /var/log/nginx/error.log warn;
pid        /var/run/nginx.pid;


events {
    worker_connections  512;
}
```

Création

- A partir d'un fichier de configuration

```
$ kubectl create configmap nginx-conf --from-file=nginx.conf
```

```
configmap/nginx-conf created
```

```
$ kubectl get configmap nginx-conf -o yaml
```

```
apiVersion: v1
data:
  nginx.conf: |
    user  nginx;
    worker_processes  auto;

    error_log  /var/log/nginx/error.log warn;
    pid        /var/run/nginx.pid;


    events {
      worker_connections  512;
    }
kind: ConfigMap
metadata:
  creationTimestamp: "2019-10-26T09:48:39Z"
```

Création

- A partir d'un fichier de variables d'environnement

```
$ cat conf.env
```

```
env=prod
log_level=debug
```

```
$ kubectl create cm config-env --from-env-file=./conf.env
```

```
configmap/config-env created
```

```
$ kubectl get cm config-env -o yaml
```

```yaml
apiVersion: v1
data:
  env: prod
  log_level: debug
kind: ConfigMap
metadata:
  creationTimestamp: "2019-09-23T20:43:35Z"
  name: config-env
  namespace: production
```

Création

- A partir d'un fichier de valeurs littérales

```
$ kubectl create cm config-lit --from-literal env=prod --from-literal log_level=debug
```

```
configmap/config-lit created
```

```
$ kubectl get cm config-lit -o yaml
```

```
apiVersion: v1
data:
  env: prod
  log_level: debug
kind: ConfigMap
metadata:
  creationTimestamp: "2019-09-23T20:43:35Z"
  name: config-lit
  namespace: production
```

Utilisation dans un Pod

- A partir d'un volume

```
$ cat cm-volume.yml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: proxy
    image: nginx:1.17
    ports:
    - containerPort: 8000
    volumeMounts:                  # montage du volume dans le conteneur
    - name: config
      mountPath: "/etc/nginx/nginx.conf"
      subPath: "nginx.conf"
  volumes:
  - name: config                   # Définition d'un volume basé sur la ConfigMap nginx-config
    configMap:
      name: nginx-conf
```

Lancement du Pod

```
$ kubectl create -f cm-volume.yml
```

```
pod/web created
```

```
$ kubectl exec -it web bash
```

```
root@web:/# more /etc/nginx/nginx.conf

user  nginx;
worker_processes  auto;

error_log  /var/log/nginx/error.log warn;
pid        /var/run/nginx.pid;


events {
    worker_connections  512;
}
```

Utilisation dans un Pod

- A partir de variable d'environnement

```
$ cat cm-env.yml
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web
spec:
  containers:
  - name: www
    image: nginx:1.17
    env:
    - name: LOG_LEVEL
      valueFrom:
        configMapKeyRef:
          name: config-env
          key: log_level
    - name: SITE
      valueFrom:
        configMapKeyRef:
          name: config-lit
          key: env
```

Lancement du Pod

```
$ kubectl create -f cm-env.yml
```

```
pod/web created
```

```
$ kubectl exec -it web bash
```

```
root@web:/# env
root@web:/# env
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
LOG_LEVEL=debug
HOSTNAME=web
PWD=/
PKG_RELEASE=1~buster
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.233.0.1:443
SITE=prod
NJS_VERSION=0.3.9
TERM=xterm
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.233.0.1
```

Secrets
secrets
CERTIFICATE

Secrets

- Protection des données
  - mots de passe
  - clés privées
- Sortir les informations sensibles des images ou des fichiers de spécification
- Plus de contrôle de leur utilisation
- Objet créé par l'utilsateur ou le système
- Utilisé dans un Pod
- Stocké dans etcd

Types

- Generic
  - fichier
  - répertoire
  - valeur litérale
- Docker-registry
  - Authentification à Registre Docker
- TLS
  - Gestion de clés

Création à partir de fichier ou littéral

```
$ cat pass.txt
```

```
password=myP@ssw.rd
```

```
$ kubectl create secret generic secret1 --from-env-file=pass.txt
```

```
secret/secret1 created
```

```
$ kubectl create secret generic secret2 --from-literal password=myP@ssw.rd
```

```
secret/secret2 created
```

```
$ kubectl get secrets
```

```
NAME                  TYPE                                  DATA   AGE
secret1               Opaque                                2      3m21s
secret2               Opaque                                2      8s
```

Secret

```
$ kubectl get secret secret1 -o yaml
```

```
apiVersion: v1
data:
  password: bXlQQHNzdy5yZA==
kind: Secret
metadata:
  creationTimestamp: "2026-04-21T06:34:35Z"
  name: secret1
  namespace: default
  resourceVersion: "209118"
  uid: 3fbdc6b8-2661-4c5e-beff-d3a3d77e2a81
```

Utilisation comme Variable d’environnement

```
$ cat redis.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
 containers:
 - name: mycontainer
   image: redis
   env:
      - name: SECRET_PASSWORD
        valueFrom:
          secretKeyRef:
            name: secret1
            key: password
```

Utilisation comme Volume

```
$ cat certs.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
  - name: mypod
    image: redis
    volumeMounts:
    - name: certs
      mountPath: "/etc/certs"
      subPath: "certs"
      readOnly: true
  volumes:
  - name: certs
    secret:
      secretName: secret3
```